

Chapter 1

Week 7 — Memory Layout & OOP Fundamentals

ENEL712 Embedded Systems Design — Lecture Week 7

Summary

Wider summary of week 7 material: program memory segmentation (stack vs heap), OOP variable vernacular (value/reference, mutability), OOP philosophy, and class components in C#.

Topics Covered

- Program memory management
- Stack vs heap comparison
- OOP variable vernacular (value/reference, mutability)
- OOP philosophy and accessibility modifiers
- Logical vs physical organisation (namespace vs assembly)
- Class components in C#

1.1 Program Memory Management

1.1.1 Memory segmentation

- High addresses: environment/arguments and the stack.
- Middle: shared libraries / DLLs.
- Low addresses: heap, .bss (uninitialised data), .data (initialised globals/statics), .text (code).

1.1.2 The stack

- LIFO region for function calls, locals, and return addresses.
- Grows downward from high to low memory.
- Very fast — simple pointer movement, cache-friendly.

- Direct access via CPU registers and the stack pointer.

1.1.3 The heap

- Used for dynamic allocation (objects, dynamic arrays).
- Grows upward toward high memory addresses.
- Manual management (new, malloc) or automatic via a Garbage Collector.
- Slower access via indirection; possible cache misses.

1.2 Stack vs Heap Comparison

Feature	Stack	Heap
Allocation	Automatic (compiler/runtime)	Manual (malloc / new) — or GC
Lifetime	Scope of the function	Until freed (or GC)
Size	Small & fixed (a few MB / thread)	Very large (limited by RAM)
Growth direction	Downward	Upward
Access	Direct via registers and SP	Indirect via pointers/references
Fragmentation	None	Possible (internal/external)
Thread safety	Per-thread	Shared; needs synchronisation
Risks	Stack overflow	Leaks, dangling pointers, double free, fragmentation

1.3 OOP Variable Vernacular

1.3.1 Value vs reference types

- Value types (int, bool, struct, enum) hold the actual data — usually on the stack; assignment copies the value.
- Reference types (class, string, array, List<T>) hold a pointer to data on the heap; assignment copies the reference, not the object.

1.3.2 Mutability

- Mutable: object can be modified after creation (e.g., List<T>); not inherently thread-safe.
- Immutable: object is fixed; modifications create new instances (e.g., string, record); naturally thread-safe.

1.3.3 Type declaration

- Explicit: type stated, e.g., string name = "Alice".
- Implicit (var): compiler infers type, e.g., var age = 30 → int.

1.4 OOP Philosophy and Accessibility Modifiers

- OOP organises programs around data and objects — modular, reusable, encapsulated, hierarchical.
- Class = template; Object = specific instance created with new.

- Encapsulation hides internal data and methods, controlling visibility.

1.4.1 Accessibility modifiers

Modifier	Scope
public	Anywhere
private	Same class only
protected	Same class and derived classes
internal	Same assembly
protected internal	Same assembly OR derived classes
private protected	Same class + derived classes in the same assembly

1.5 Logical vs Physical Organisation

Aspect	Namespace	Assembly
Purpose	Logical organisation; prevents naming collisions	Physical packaging and deployment unit
Concept	Compile-time (logical)	Runtime (physical)
What it is	Naming container for types	Compiled file (.exe / .dll) with IL and metadata

1.6 Class Components in C#

- Properties / fields: data members (e.g., `account.Balance`); best practice is to keep them private.
- Accessors (get/set): controlled access; set uses the implicit value parameter.
- Methods: functions; only public methods are visible outside.
- Constructors: same name as class, no return type; allocate and initialise an object on new.
- Overloading: multiple methods/constructors with same name and different parameter signatures.

1.7 Use Cases & Applications

- Stack-vs-heap intuition guides daily C# decisions: prefer struct + value semantics for tiny, immutable data, class for anything large or mutable.
- Mutability rules are the difference between safe-by-default code (immutable strings, records) and code that needs locks to be thread-safe.
- Access-modifier choices document intent — protected on a base member tells future readers 'this is meant to be reused only via inheritance'.
- Namespace vs assembly thinking matters for deployment: a NuGet package is one assembly; the namespaces inside organise its API.

1.8 Worked Example: Stack Overflow Demonstration

```
// In C# (or C), unbounded recursion is the fastest way to break the stack.
void Recurse(int depth)
{
    int[] big = new int[1024];    // each call grabs ~4 KB of stack space
    Recurse(depth + 1);
}
Recurse(0);                      // crashes with StackOverflowException
```

On embedded systems the situation is worse: no exception, just silent corruption — a strong reason to budget stack and avoid recursion.

1.9 Mutability Cheat Sheet

Type	Mutable?	Where stored
string	No	Heap
int / double / bool	Yes (value)	Stack / inline
DateTime	No	Stack (struct)
List<T>	Yes	Heap
Tuple (record struct)	No	Stack

1.10 Practical Notes

- Treat fields as private by default. Promote to internal/protected only when you have a use case.
- Use readonly fields and { get; } properties to make immutability explicit.
- Don't force everything into a single namespace; group classes the way callers think about them.
- An assembly is the unit of versioning and shipping; a namespace is just a naming convenience inside it.

1.11 Lecture Notes: Memory Layout in Practice

Process memory has four logical regions. Globals split between BSS (uninitialised) and DATA (initialised). The stack grows downward from a high address; the heap grows upward from a low address. The free space between them is what either side can grow into.

Region	Direction	Lifetime	Typical contents
BSS (uninitialised data)	Fixed at link time	Whole program	Globals declared but not assigned a value
DATA (initialised data)	Fixed at link time	Whole program	Globals/static with non-zero initialiser
Stack	Grows down from high memory	Per function call	Locals, parameters, return addresses
Heap	Grows up from low memory	Until freed / GC'd	new objects, dynamically sized buffers

- When the stack and heap meet, the system fails — classic stack overflow.

- Globals also have a fixed budget; using too many of them eats space that could otherwise be free for stack/heap growth.
- Treat 'where does this live?' as a first-class design question, not an implementation detail.

1.12 Stack Mechanics — Push / Pop / Stack Pointer

- Calling a function pushes a frame onto the stack: parameters, locals, and the return address.
- Returning pops the frame automatically — you almost never pop manually in high-level code.
- The Stack Pointer (SP) always sits at the current top; pushing a new frame moves SP towards lower memory.
- Nested calls grow the stack: $\text{main} \rightarrow \text{A} \rightarrow \text{B}$ leaves three frames live at once.
- Because pop is automatic, stack memory management is essentially free — that is its single biggest advantage.

1.12.1 Worked example: stack frames during nested calls

Step	What happens	Stack frames (top $\rightarrow$ bottom)
1	Program starts	main
2	main calls A	A, main
3	A calls B	B, A, main
4	B returns to A	A, main
5	A returns to main	main

1.13 Recursion — Why Embedded Engineers Avoid It

A recursive function calls itself. Every call adds another frame to the stack; the deeper the recursion, the larger the stack grows. With no termination, the stack collides with the heap and the program crashes — a stack overflow.

- Tree/graph algorithms feel natural recursive — but on an MCU you almost never have the stack budget.
- Prefer explicit iteration with a heap-allocated worklist; it is bounded and inspectable.
- If you must recurse, prove an upper bound on depth and add a safety counter.

```
// Bad: unbounded recursion -- stack grows on every call
int Sum(int n) {
    if (n == 0) return 0;
    return n + Sum(n - 1);    // 10,000 deep  $\Rightarrow$  stack blows up
}

// Good: iterative -- single frame, constant stack usage
int Sum(int n) {
    int total = 0;
    for (int i = 1; i <= n; i++) total += i;
    return total;
}
```

1.14 Heap Behaviour — Fragmentation, GC, malloc

- The heap is unordered — allocations stay where placed until explicitly freed (C/C++) or collected (C#/Java).
- Repeated allocate/free cycles cause fragmentation: lots of small free holes but no contiguous block big enough for the next request.
- C and C++ expose malloc/free directly; C# hides this behind the .NET runtime and its garbage collector.
- The garbage collector finds heap objects no longer referenced by any live variable and reclaims them; modern GCs also compact the heap to defragment.
- Class instances in C# always live on the heap; their reference (the variable) lives on the stack.
- When the reference goes out of scope, the heap object becomes unreferenced and is eligible for collection.

1.15 Threading and the Stack

- Each thread has its own stack — local variables are inherently thread-safe.
- All threads share the heap — accesses to shared objects need synchronisation (lock, Interlocked, immutability).
- True parallelism requires multiple cores; a single core interleaves threads (often described colloquially as 'parallel').
- On a microcontroller, multi-threading is rare — the stack budget alone is usually prohibitive.

1.16 Stack vs Heap — One-Page Comparison

Aspect	Stack	Heap
Allocation	Automatic on call	Manual (or via new + GC)
Speed	Fast — pointer bump	Slower — bookkeeping, search
Lifetime	Scope of the call	Until freed / collected
Order	LIFO	Free-for-all
Direction	Grows down from high mem	Grows up from low mem
Thread safety	Per-thread, inherently safe	Shared, needs synchronisation
Fragmentation	None	Possible — GC compacts in C#
Typical contents	Locals, parameters, return addresses	Objects allocated with new, large buffers

Key Definitions

Stack

LIFO memory region for function calls, locals, and temporary data.

Heap

Region for dynamic allocation; managed manually or via a GC.

Garbage Collector

Automatic memory reclamation system for unreachable heap objects.

Value Type

Holds the actual data (int, struct); usually on the stack.

Reference Type

Holds an address to data on the heap (class, array, string).

Mutable

Object whose state can be changed after creation.

Immutable

Object whose state is fixed once created.

Class

Template for objects, including data and methods.

Object

Instance of a class created with new.

Encapsulation

Bundling data and methods and restricting external access.

Namespace

Compile-time logical container preventing naming collisions.

Assembly

Runtime physical packaging unit (.dll / .exe).

Constructor

Method invoked on creation; allocates and initialises fields.

Overloading

Multiple methods/constructors with same name and different signatures.

Accessor

get / set methods providing controlled access to private properties.

Stack Frame

Block of stack memory pushed for one function call: parameters, return address, locals, saved registers.

Memory Leak

Heap memory that remains allocated because something still references it, but is never used again.

Dangling Pointer

Pointer to memory that has been freed; classic C bug that GC eliminates in C#.

Thread-Safe

Behaves correctly when called concurrently from multiple threads.

Stack Pointer (SP)

Register that always points to the current top of the stack; updated on every push or pop.

Stack Overflow

Failure caused when the stack grows so large it collides with the heap or other memory regions.

Fragmentation

Heap state where free memory is split into many small holes, blocking large contiguous allocations.

Garbage Collector

.NET runtime service that reclaims heap objects no longer referenced by live code, optionally compacting the heap.

malloc / free

C/C++ standard-library functions for explicit heap allocation and deallocation; equivalent to new/delete in C++.

Sources

- Lecture Week 7.pdf